

— ECE 3301 —

Microcontrollers with Drone Examples

*PIC18F Microcontroller Concepts
Applied to Real-World Drone Applications*



Lab 4: Drone Mission Timing and Safety Supervisor

ECE 3301 – Microcontrollers | Fall 2026

Instructor: Sasoun Torousian

ECE 3301L – Lab 4

Drone Mission Timing and Safety Supervisor

PIC18F46K22 + Curiosity HPC

1. Introduction

In the previous labs, you learned how to configure digital inputs and outputs, read pushbuttons, control LEDs, make decisions using C, and implement basic drone safety logic. You have also learned that a real embedded system cannot simply stop everything whenever it needs to wait. A drone flight controller must perform many operations at the same time. It may need to check for faults continuously, blink a status indicator every few hundred milliseconds, verify an ARM command for several seconds, monitor whether communication has been lost, and update other subsystems at different rates. In this lab, you will build your first simple real-time drone supervisor. The objective is not simply to blink LEDs using a timer. You will use one hardware timer to create a common system time base and use that time base to coordinate several independent drone functions.

Your controller will have to:

- monitor an ARM command,
- monitor an emergency FAULT input,
- qualify the ARM command for a specified amount of time,
- generate different LED patterns depending on the operating state,
- detect a simulated communication timeout,
- automatically return to a safe condition when communication is lost,
- keep monitoring safety inputs while all timing operations are occurring.

The important challenge is that all of these operations must happen without long blocking delays. You may not solve the lab by placing `__delay_ms()` calls throughout the program. By the end of this lab, your PIC18F46K22 will behave much more like a small flight-controller supervisor than a simple microcontroller demonstration.

2. What You Are Building

Imagine that your PIC18F46K22 is responsible for supervising a drone before and during operation. It does not directly control the motors yet. Instead, it decides whether propulsion would be permitted and continuously checks whether the system remains safe. Your controller has four operating states:

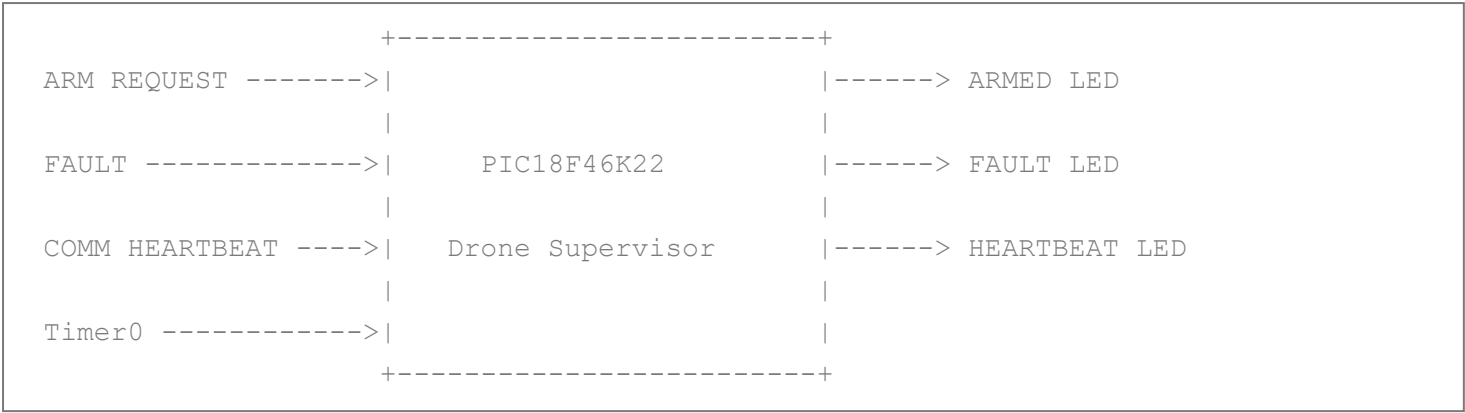
SAFE

ARMING

ARMED

FAULT

The system will also monitor a simulated communication heartbeat from another device. A pushbutton will represent communication packets arriving from a future radio receiver or flight computer. The overall system is:



Timer0 will provide the timing reference for the entire system.

3. Learning Objectives

After completing this lab, you should be able to configure Timer0 as a hardware timer, calculate a timer preload value, detect a timer overflow by polling the timer flag, generate a periodic system tick, use software counters to create several timing intervals from one hardware timer, combine timing with a state machine, implement non-blocking timing, prioritize a safety fault over normal operation, and explain why this architecture is more useful than using long software delays.

4. Required Hardware

Use the following hardware:

Quantity	Component	Purpose
1	Curiosity HPC with PIC18F46K22	Main controller
3	Pushbuttons	ARM, FAULT, and COMM heartbeat
3	LEDs	ARMED, FAULT, and HEARTBEAT
3	220 Ω to 470 Ω resistors	LED current limiting

Quantity	Component	Purpose
As needed	Jumper wires	Connections
1	Breadboard	External circuit

You may use the PORTB internal pull-ups for the pushbuttons if configured correctly.

5. Input and Output Assignment

Use the following pin assignments unless instructed otherwise.

Signal	PIC Pin	Direction	Purpose
ARM REQUEST	RB0	Input	Requests arming
FAULT	RB1	Input	Simulates a critical drone fault
COMM HEARTBEAT	RB2	Input	Simulates communication from another controller
ARMED LED	RD0	Output	Indicates propulsion permission
FAULT LED	RD1	Output	Indicates fault condition
HEARTBEAT LED	RD2	Output	Indicates firmware operation

Remember the GPIO rules you learned previously:

```
ANSELx → configure digital operation
TRISx  → configure input/output direction
PORTx  → read an input
LATx   → control an output
```

The PIC GPIO lecture specifically distinguishes reading physical inputs through `PORTx` and controlling outputs through `LATx`. :chatgpt-content-reference{index="3"}

6. Pushbutton Wiring

Configure RB0, RB1, and RB2 using active-low pushbuttons. Each input should normally be HIGH and become LOW when its button is pressed. Conceptually:

```
VDD
|
Internal Pull-Up
|
+----- RBx
|
Pushbutton
|
GND
```

Therefore:

```
Button released → PORTB bit = 1
Button pressed  → PORTB bit = 0
```

Do not leave an input floating. A floating digital input does not have a guaranteed logic state and may randomly appear HIGH or LOW. :chatgpt-content-reference{index="4"}

7. LED Wiring

Connect the three LEDs to RD0, RD1, and RD2 through individual current-limiting resistors.

For this lab, assume:

```
LATD bit = 1 → LED ON
LATD bit = 0 → LED OFF
```

The three LEDs have different meanings.

```
RD0 → ARMED LED
RD1 → FAULT LED
RD2 → HEARTBEAT LED
```

8. Required System Behavior

Your controller must begin in a safe condition every time power is applied or the PIC is reset.

At startup:

```
State           = SAFE
ARMED LED       = OFF
FAULT LED       = OFF
HEARTBEAT LED   = OFF
```

The propulsion-permission output must never become active automatically after reset. Safe initialization is an important embedded-system principle, and the drone-controller lecture requires outputs to enter known safe conditions during startup.

9. Timer0 System Tick

Configure Timer0 to generate a timing event every approximately:

```
100 ms
```

Use:

```
FOSC = 16 MHz
Timer0 = 16-bit mode
Clock source = internal instruction clock
Prescaler = 1:8
```

10. Important Restriction

You are not using interrupts in this lab. You must detect Timer0 overflow by polling:

```
if(INTCONbits.TMR0IF == 1)
{
    // A 100 ms system tick occurred
}
```

When an overflow occurs, you must reload Timer0 and clear the overflow flag. Conceptually:

```
Main Loop
|
+---- Read Inputs
|
+---- Check Fault
|
+---- Execute State Machine
|
+---- Has Timer0 overflowed?
      |
      YES
      |
      +---- Reload Timer0
      +---- Clear TMR0IF
      +---- Update timing counters
```

Do not wait inside the main loop for Timer0 to overflow. The main loop should continue running.

11. The System Tick

Every Timer0 overflow represents:

```
1 tick = 100 ms
```

This gives you a simple time reference. For example:

```
5 ticks = 500 ms
10 ticks = 1 second
20 ticks = 2 seconds
30 ticks = 3 seconds
```

You will use separate software counters for different jobs. For example:

```
uint8_t heartbeatCounter = 0;
uint8_t armCounter = 0;
uint8_t commCounter = 0;
```

Do not use one counter for everything. These timers represent different events and must be able to operate independently.

12. Required State Machine

Your controller must implement:

```
SAFE
ARMING
ARMED
FAULT
```

You may define the states using constants:

```
#define SAFE_STATE      0
#define ARMING_STATE    1
#define ARMED_STATE     2
#define FAULT_STATE     3
```

and create:

```
uint8_t systemState = SAFE_STATE;
```

The state-machine approach is important because the behavior of the same input depends on the current operating condition. Your drone lecture introduces this specifically to avoid building increasingly complicated firmware from unrelated `if` statements. :chatgpt-content-reference{index="7"}

13. SAFE State

When the controller is SAFE:

```
ARMED LED = OFF  
FAULT LED = OFF
```

The system remains SAFE until an ARM request is detected.

If ARM is pressed:

```
SAFE → ARMING
```

When entering ARMING:

```
armCounter = 0
```

Do not immediately arm the system.

14. ARMING State

The ARM button must remain continuously pressed for:

```
2.0 seconds
```

Because:

```
1 tick = 100 ms
```

you need:

```
20 ticks
```

During ARMING, increment `armCounter` only when a new 100 ms system tick occurs.

If:

```
armCounter >= 20
```

and the ARM button is still pressed and no fault exists:

```
ARMING → ARMED
```

If ARM is released before two seconds have elapsed:

```
ARMING → SAFE
```

and:

```
armCounter = 0
```

This requirement means a quick accidental button press cannot arm the drone.

15. ARMED State

When the system reaches ARMED:

```
ARMED LED = ON  
FAULT LED = OFF
```

The controller must remain responsive. Releasing the ARM button must immediately return the system to:

```
SAFE
```

Therefore:

```
ARMED + ARM released → SAFE
```

This is different from simply waiting for another timed event. Safety inputs should be evaluated continuously by the main loop.

16. Firmware Heartbeat

RD2 is the firmware heartbeat LED. The heartbeat proves that the main program is continuing to execute. The heartbeat pattern must depend on the drone state. Use the following requirements:

State	Heartbeat Behavior
SAFE	Toggle every 500 ms
ARMING	Toggle every 200 ms
ARMED	Toggle every 1 second

State	Heartbeat Behavior
FAULT	OFF

Since the system tick is 100 ms:

```
SAFE      → toggle every 5 ticks
ARMING    → toggle every 2 ticks
ARMED     → toggle every 10 ticks
```

This means a person looking at the controller should be able to determine the approximate operating state without pressing any buttons. This requirement also forces you to use the same timer for several different timing behaviors.

17. Emergency Fault Requirement

The FAULT input has the highest priority in the entire system. A fault may occur while the system is:

```
SAFE
ARMING
ARMED
```

If the FAULT button is pressed:

```
ANY STATE → FAULT
```

Immediately:

```
ARMED LED = OFF
FAULT LED = ON
HEARTBEAT LED = OFF
```

Do not wait for the next 100 ms timer tick before removing the ARMED indication. The fault must be detected by the normal high-speed execution of the main loop. The drone architecture introduced in lecture makes this priority explicit: a fault must remove propulsion permission even if the ARM request remains active.

18. Recovering From FAULT

Releasing the FAULT button must not automatically return the controller to ARMED. That would be unsafe. The controller may return from FAULT to SAFE only when:

```
FAULT released
AND
ARM released
```

Therefore:

```
FAULT
|
| FAULT cleared AND ARM released
v
SAFE
```

The operator must perform the complete arming sequence again before ARMED can be reached.

19. Communication Heartbeat

Now you will add the most challenging part of the lab. RB2 represents a communication message arriving from another controller. Imagine that another processor or radio normally sends a message to the flight controller periodically. In this lab, pressing the COMM button represents receiving one of these messages. Every time the COMM button is pressed:

```
commCounter = 0;
```

The controller therefore knows:

```
"I recently heard from the communication system."
```

When the system is ARMED, `commCounter` must increase once every 100 ms. For example:

```
0.0 s → commCounter = 0
0.1 s → commCounter = 1
0.2 s → commCounter = 2
...
```

You are now creating a simple communication watchdog.

20. Communication Timeout

While ARMED, the controller must receive at least one COMM heartbeat every:

```
3 seconds
```

Because:

```
3 seconds / 100 ms = 30 ticks
```

if:

```
commCounter >= 30
```

the controller must assume communication has been lost. The required response is:

```
ARMED → FAULT
```

Immediately:

```
ARMED LED = OFF  
FAULT LED = ON
```

This creates a realistic drone behavior. If communication with the pilot or another critical controller disappears, the system should not simply assume that everything is okay. For this lab, the COMM button is only a simulation. Later in the course, communication may arrive through UART, SPI, I²C, or another interface.

21. Communication Timer Rules

The communication timeout only applies while the system is ARMED. When entering ARMED:

```
commCounter = 0
```

Every valid COMM button press while ARMED resets:

```
commCounter = 0
```

Every 100 ms tick while ARMED increments:

```
commCounter++
```

If:

```
commCounter >= 30
```

then:

```
systemState = FAULT_STATE;
```

When the system is not ARMED, reset:

```
commCounter = 0
```

22. Button Edge Requirement

There is one additional challenge. Holding the COMM button down must not continuously reset the communication timer. A communication event should occur once when the button transitions from released to pressed. Therefore, you must detect:

```
Released → Pressed
```

rather than simply:

```
Button is currently pressed
```

You may create variables such as:

```
uint8_t commNow;  
uint8_t commPrevious;
```

Conceptually:

```
Previous = RELEASED  
Current  = PRESSED  
  
      ↓  
  
New communication event
```

After processing the inputs:

```
commPrevious = commNow
```

This is called edge detection. You are not using an interrupt for this yet. You are detecting the edge through polling. This detail is important because Lab 5 will revisit exactly this type of event using interrupts.

23. Required Timing Architecture

Your program must use one Timer0 system tick to generate all of the following timing behaviors:

```
Timer0  
|  
+----- 100 ms System Tick  
|
```

```
+----- ARM qualification
|         20 ticks = 2 seconds
|
+----- Communication timeout
|         30 ticks = 3 seconds
|
+----- SAFE heartbeat
|         5 ticks = 500 ms
|
+----- ARMING heartbeat
|         2 ticks = 200 ms
|
+----- ARMED heartbeat
|         10 ticks = 1 second
```

Do not configure a separate hardware timer for every requirement. Part of the engineering challenge is learning how one periodic time base can coordinate several software tasks.

24. Recommended Program Structure

Your final program should have a structure similar to:

```
main()
|
+----- Configure oscillator
|
+----- Configure GPIO
|
+----- Configure internal pull-ups
|
+----- Configure Timer0
|
+----- Initialize state and counters
|
+----- while(1)
|       |
|       +----- Read ARM
|       |
|       +----- Read FAULT
|       |
|       +----- Read COMM
|       |
|       +----- Detect COMM edge
|       |
|       +----- Check critical FAULT condition
|       |
|       +----- Execute state machine
```

```

|
+----- Check Timer0 overflow
|       |
|       +----- Reload Timer0
|       +----- Clear TMR0IF
|       +----- Update counters
|
+----- Check communication timeout
|
+----- Update LEDs
|
+----- Repeat

```

You do not have to use exactly this software organization, but your program must satisfy all functional requirements.

25. Suggested Functions

You are encouraged to organize your program using functions such as:

```

void oscillator_init(void);
void gpio_init(void);
void timer0_init(void);
void read_inputs(void);
void process_state_machine(void);
void process_system_tick(void);
void update_outputs(void);

```

Do not create functions simply to make the program look complicated. Each function should have a clear responsibility.

26. What You Must Figure Out

Unlike the earlier labs, you are not being given the complete program. You must determine how to organize the state machine, how to maintain several independent timing counters, when each counter should increment, when each counter should reset, how to prevent a held COMM button from generating repeated events, how to guarantee that FAULT has priority, and how to keep the entire system responsive without blocking delays. You already have the individual concepts required to solve the problem. The challenge is combining them into one working embedded system.

27. Restrictions

Your final program must not use long blocking delays to implement the required behavior. Specifically, you may not solve the timing requirements using code such as:

```
__delay_ms(2000);  
__delay_ms(3000);  
__delay_ms(500);
```

You must use Timer0 for the timing requirements. You must not use interrupts. Timer0 overflow must be detected by polling `TMR0IF`. You must implement the four required states. You must implement the communication timeout. You must implement COMM button edge detection.

28. Required Demonstration

- During the laboratory demonstration, you must demonstrate the complete system to the instructor.
- First, reset the PIC. Show that the controller starts in SAFE with the ARMED and FAULT LEDs off.
- Next, briefly press ARM for less than two seconds. Show that the controller enters ARMING but returns to SAFE when ARM is released. The drone must not become ARMED.
- Next, press and continuously hold ARM for at least two seconds. Show that the controller enters ARMED and the ARMED LED turns on.
- Show the different heartbeat rates in SAFE, ARMING, and ARMED.
- While ARMING, press FAULT. Show that the system immediately enters FAULT without waiting for the two-second arming timer to finish.
- Recover correctly by releasing both FAULT and ARM and show that the system returns to SAFE rather than automatically returning to ARMED.
- Arm the system again. While ARMED, press COMM periodically at intervals shorter than three seconds. Show that the controller remains ARMED.
- Arm the system again and stop pressing COMM. After approximately three seconds, show that the controller detects communication loss and enters FAULT.
- Finally, hold the COMM button continuously. Demonstrate that holding the button does not count as repeated communication events. The controller should eventually detect a communication timeout because only the initial press represents a new communication event.

29. What to Submit

- Submit your complete MPLAB X project or source code as instructed by your instructor.
- No formal laboratory report is required for this lab.
- Your code must be readable and appropriately commented. Comments should explain why important operations are performed rather than simply translating each C statement into English.